

International Burch University

IT 506 — Software Verification, Validation and Testing

Course Project Report

Verification, Validation & Testing of a MERN E-Commerce Platform
ProShop

Author: Hena Potogija

hena.potogija@stu.ibu.edu.ba

June 2026

Table of Contents

1. Introduction	4
1.1. About the Project	4
1.2. Deliverable Links	4
1.3. Project Functionalities and Screenshots	4
1.3.1. Storefront / Home Page	4
1.3.2. Product Search & Pagination	5
1.3.3. Product Details & Reviews	5
1.3.4. Shopping Cart	6
1.3.5. User Authentication	6
1.3.6. Checkout (PayPal Sandbox)	6
1.3.7. Order History	7
1.3.8. Admin Area	7
2. Test Plan	9
2.1. Scope	9
2.2. Testing Strategy Overview	9
2.3. Testing Environment and Tools	9
3. Static Analysis	11
3.1. Initial Run — 8 problems (2 errors, 6 warnings)	11
3.2. Final Run — 1 problem (0 errors, 1 warning)	11
3.3. Security Finding	11
4. Test Case Design	12
4.1. Unit — calcPrices() pricing logic	12
Equivalence partitioning	12
Boundary value analysis (threshold = 100)	12
Decision table (free-shipping rule)	12
4.2. Integration — HTTP API test cases	12
4.3. System (E2E) — Playwright user journeys	13
5. Test Execution	14
5.1. Unit Testing	14
5.2. Integration Testing	14
Product API	14
User / Auth API	15
5.3. System (End-to-End) Testing	15
6. Test Coverage Analysis	16
7. Bug Reports and Fixes	17
7.1. BUG-001 — Product search crashes on regex metacharacters (regex injection)	17
Description	17
Steps to reproduce	17
Fix	17

Verification	17
7.2. Code-quality fix — password re-hash on profile update	17
8. Conclusion	18
8.1. Testing Summary	18
8.2. Lessons Learned and Final Thoughts	18

1. Introduction

1.1. About the Project

ProShop is an open-source e-commerce platform built on the MERN stack (MongoDB, Express, React, and Node.js). It implements a complete online storefront: a searchable, paginated product catalogue, product detail pages with customer reviews, a shopping cart, JWT-cookie based authentication, a multi-step checkout backed by the PayPal sandbox, order history, and an administrative area for managing products, users, and orders.

This project does not aim to build a new application from scratch; instead, it applies the verification, validation, and testing techniques taught in IT 506 to a real, non-trivial, publicly available codebase. The original application was forked from Brad Traversy's open-source repository and extended with a full testing stack: static analysis, unit testing, integration testing, system (end-to-end) testing, coverage analysis, a documented bug fix accompanied by a regression test.

The application is non-trivial in both size and functionality — it spans a Node/Express REST API, a React single-page front end, a MongoDB data layer, authentication, and third-party payment integration — which makes it a realistic target for the full spectrum of verification and validation activities required by the course.

1.2. Deliverable Links

Deliverable	Link
GitHub repository	https://github.com/HenaPot/ProShop_SVVT
Hosted application (Render)	https://proshop-svvt.onrender.com
Original application (upstream)	https://github.com/bradtraversy/proshop-v2 (MIT License)

The repository contains the original source code, all modifications and testing artifacts, and a detailed README with run and test instructions. The original work by Brad Traversy is preserved and credited under its MIT License (see *LICENSE* and *docs/ORIGINAL_PROSHOP_README.md* in the repository).

1.3. Project Functionalities and Screenshots

The following sections illustrate the core functionalities of the system under test.

1.3.1. Storefront / Home Page

The landing page lists the product catalogue with images, prices, ratings, and pagination. It is the entry point from which users browse, search, and navigate to product details.

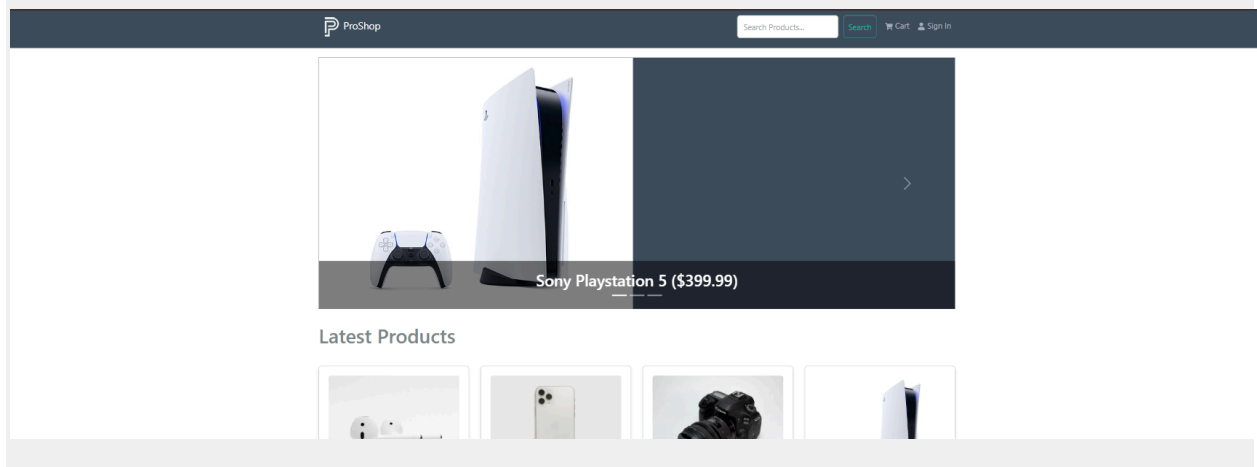


Figure: Storefront / Home Page in the running ProShop application.

1.3.2. Product Search & Pagination

Users can search the catalogue by keyword. Results are filtered case-insensitively by product name and paginated, allowing users to quickly find the products they are looking for.

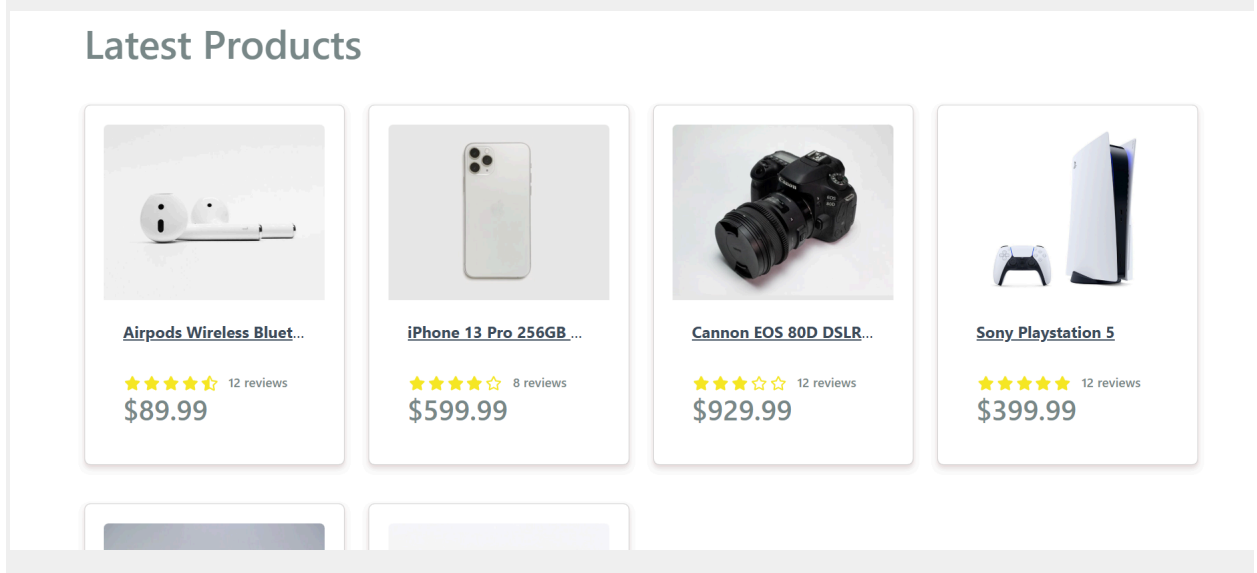


Figure: Product Search & Pagination in the running ProShop application.

1.3.3. Product Details & Reviews

Each product has a detail page showing description, price, stock availability, an “Add to Cart” control, and customer reviews and ratings.

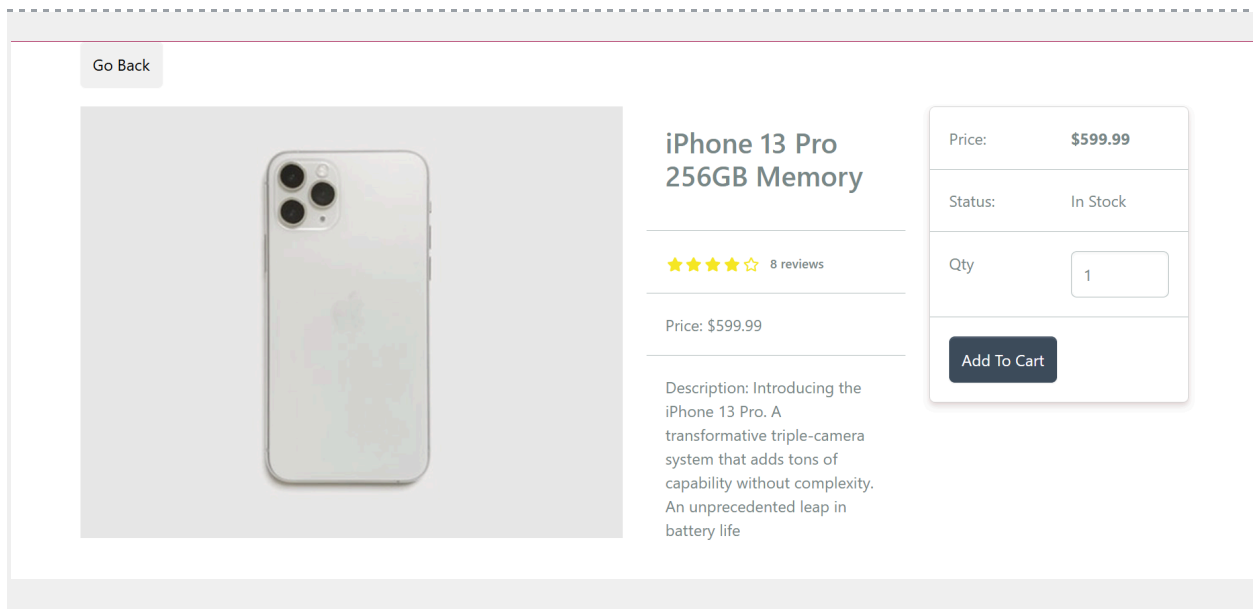


Figure: Product Details & Reviews in the running ProShop application.

1.3.4. Shopping Cart

Users add items to the cart, adjust quantities, and remove items. The cart shows line items and a running subtotal in preparation for checkout.

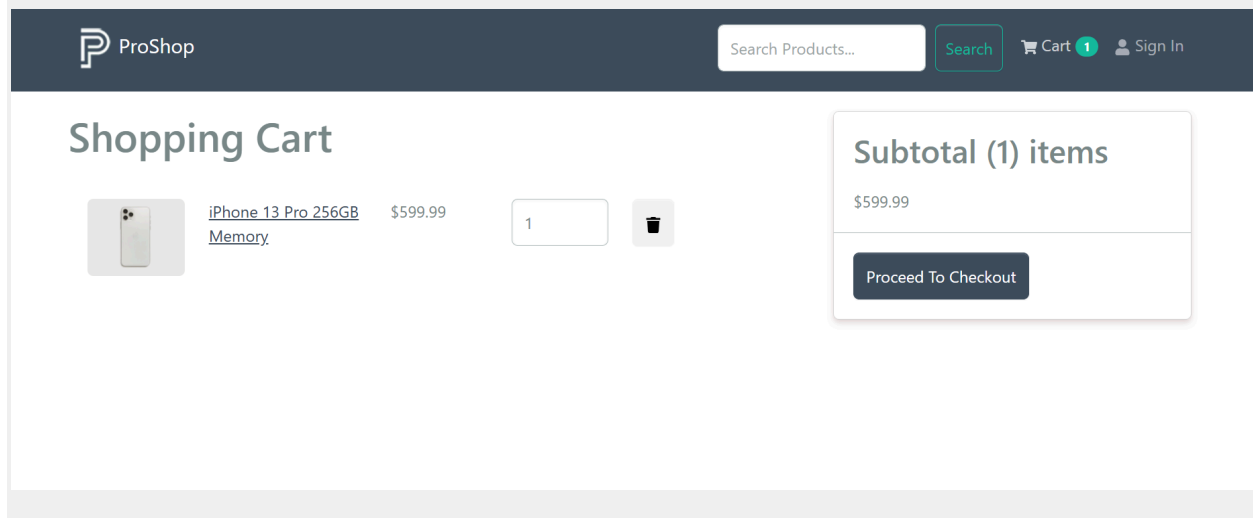


Figure: Shopping Cart in the running ProShop application.

1.3.5. User Authentication

Registration and login are handled with a JWT stored in a hardened, HTTP-only cookie. Authentication protects user data and gates access to profile and checkout features.

1.3.6. Checkout (PayPal Sandbox)

A multi-step checkout collects shipping and payment details and processes payment through the PayPal sandbox environment — no real charges are ever made.

ProShop

Search Products... Search Cart test

Sign In Shipping Payment Place Order

Shipping

Address: test, test, test

Payment Method

Method: PayPal

Order Items

1 x iPhone 13 Pro 256GB Memory	1 x \$599.99 = \$599.99
--------------------------------	-------------------------

Order Summary

Items	\$599.99
Shipping	\$0.00
Tax	\$90.00
Total	\$689.99

Place Order

Figure: Checkout (PayPal Sandbox) in the running ProShop application.

1.3.7. Order History

Authenticated users can view their past orders and order details, including payment and delivery status.

ProShop

Search Products... Search Cart test

Order 6a2f1783499d702909caf4ab

Shipping

Name: test
Email: test@gmail.com
Address: test, test, test

Not Delivered

Payment Method

Method: PayPal

Not Paid

Order Items

1 x iPhone 13 Pro 256GB Memory	1 x \$599.99 = \$599.99
--------------------------------	-------------------------

Order Summary

Items	\$599.99
Shipping	\$0
Tax	\$90
Total	\$689.99

Figure: Order History in the running ProShop application.

1.3.8. Admin Area

Administrators manage products, users, and orders — creating, editing, and deleting catalogue items and updating order fulfilment status.

 ProShop

[Search](#)

[Cart](#)

[Admin User](#)

[Admin](#)

User Profile

Name

Email Address

Password

Confirm Password

[Update](#)

My Orders

ID	DATE	TOTAL	PAID	DELIVERED
----	------	-------	------	-----------

Figure: Admin Area in the running ProShop application.

2. Test Plan

2.1. Scope

The testing effort covers the application's critical business logic and user journeys across all verification and validation levels. The following areas are in scope:

- Order pricing logic — items subtotal, free-shipping threshold, tax, and total calculation.
- Authentication — token generation, registration, login, and access control on protected routes.
- Product API — listing, keyword search, pagination, single-product retrieval, and error handling.
- Core storefront journeys — browsing, searching, viewing a product, adding to / removing from the cart, registering, and logging in.
- Code quality — static analysis of the backend source for defects and risky constructs.
- Security — input handling on the search endpoint (regex-injection vector, BUG-001).

Out of scope:

- Real online payments — checkout uses the PayPal sandbox only; no live financial transactions are tested.
- Exhaustive UI styling / pixel-level visual regression of the React front end.
- Load and performance testing, which is beyond the scope of this course project.

2.2. Testing Strategy Overview

A layered strategy was applied so that defects are caught at the most appropriate level, from static inspection of the source up to black-box journeys against the deployed system.

Level	Technique	Tool	Location
Static analysis	Linting / code review	ESLint 9	docs/static-analysis.md
Unit	BVA, equivalence partitioning, decision table	Vitest	backend/__tests__/unit
Integration	API contract, auth, error paths	Vitest + Supertest + in-memory MongoDB	backend/__tests__/integration
System (E2E)	Black-box user journeys	Playwright	tests/e2e
Coverage	Statement / branch coverage	Vitest + v8	coverage/
Regression	Re-runnable suites	Playwright	tests/e2e

2.3. Testing Environment and Tools

Environment:

- Node.js 20 runtime.
- MongoDB (MongoDB Atlas for the running app; in-memory MongoDB for integration tests).
- Visual Studio Code as the development environment; npm scripts to run every activity.

Tools and frameworks:

- **ESLint 9 (flat config)** — Static analysis of the backend without executing it.
- **Vitest** — Unit and integration test runner.
- **Supertest** — HTTP-level assertions against the Express app for integration tests.
- **mongodb-memory-server** — Throwaway in-memory MongoDB so integration tests never touch the real database.
- **Playwright** — Black-box system tests across desktop and mobile viewports.
- **@vitest/coverage-v8** — Statement and branch coverage measurement.

3. Static Analysis

Static analysis inspects the source code **without executing it**, flagging unused code, undeclared globals, unreachable statements, and risky constructs. The backend was analysed with ESLint 9 using a flat configuration (*eslint.config.js*) and run with `npm run lint`.

3.1. Initial Run — 8 problems (2 errors, 6 warnings)

File	Rule	Finding	Resolution
utils/paypal.js (×2)	no-undef	"fetch" not defined	False positive — fetch is a Node 18+ global; added to ESLint globals.
middleware/errorMiddleware.js	prefer-const	statusCode, message use let	Auto-fixed (--fix) to const.
middleware/authMiddleware.js	prefer-const	token never reassigned	Merged declaration + assignment into const token = req.cookies.jwt.
seeder.js	no-unused-vars	mongoose imported but unused	Removed the dead import.
seeder.js	no-unused-vars	colors import unused	Changed to a side-effect import (patches String.prototype).
middleware/errorMiddleware.js	no-unused-vars	next unused	Intentional — Express identifies an error handler by its 4-argument arity, so the parameter must stay. Documented, justified warning.

3.2. Final Run — 1 problem (0 errors, 1 warning)

```
backend/middleware/errorMiddleware.js
  7:38  warning  'next' is defined but never used  no-unused-vars
```

✖ 1 problem (0 errors, 1 warning)

`npm run lint` exits with code 0 (ESLint fails the build only on errors). The single remaining warning is the justified Express error-handler signature described above.

3.3. Security Finding

The static-analysis pass also surfaced that user input flowed into a MongoDB \$regex query unescaped. This was escalated to a tracked defect — BUG-001 (regex injection) — documented and fixed in Section 7 with an accompanying regression test.

```
C:\Master\SVVT\ProShop_SVVT>npm run lint

> proshop2@2.0.0 lint
> eslint backend

C:\Master\SVVT\ProShop_SVVT\backend\middleware\errorMiddleware.js
  7:38  warning  'next' is defined but never used  no-unused-vars

✖ 1 problem (0 errors, 1 warning)
```

Figure: ESLint static-analysis output showing the final single justified warning.

4. Test Case Design

Test cases were designed with established black-box techniques — equivalence partitioning, boundary value analysis (BVA), and decision tables — for the pricing logic, and with contract / error-path analysis for the API and user-journey analysis for the system tests.

4.1. Unit — calcPrices() pricing logic

Rules under test: $\text{shipping} = (\text{itemsPrice} > 100) ? 0 : 10$; $\text{tax} = 15\% \times \text{itemsPrice}$; $\text{total} = \text{items} + \text{shipping} + \text{tax}$.

Equivalence partitioning

Partition	Representative input	Expected shipping
Empty cart	[] (items = 0)	10
Below threshold	items = 50	10
Above threshold	items = 150	0

Boundary value analysis (threshold = 100)

itemsPrice	Expected shipping	Note
99.99	10	just below
100.00	10	on boundary — rule is strictly > 100
100.01	0	just above

Decision table (free-shipping rule)

Condition: itemsPrice > 100	Action: shippingPrice
F (false)	10
T (true)	0

4.2. Integration — HTTP API test cases

#	Endpoint	Scenario	Expected
P1	GET /api/products	List seeded products	200, paginated {products, page, pages}
P2	GET /api/products?keyword=	Case-insensitive name filter (hit / miss)	200, filtered
P3	GET /api/products?keyword=(	Regex metacharacter (BUG-001)	200, no crash
P4	GET /api/products/:id	Valid id	200, single product
P5	GET /api/products/:id	Invalid ObjectId	404
P6	GET /api/products/:id	Valid-but-missing id	404 "not found"
P7	POST /api/products	Unauthenticated create	401
U1	POST /api/users	Register new user	201, jwt cookie, no password leaked
U2	POST /api/users	Duplicate email	400 "already exists"
U3	POST /api/users/auth	Correct credentials	200

#	Endpoint	Scenario	Expected
U4	POST /api/users/auth	Wrong password	401
U5	GET /api/users/profile	No token	401 "not authorized"
U6	PUT /api/users/profile then re-login	Password integrity after update	200 (guards userModel fix)

4.3. System (E2E) — Playwright user journeys

#	Journey	Expected
E1	Home loads, products visible	Header + product cards render
E2	Search by keyword	Results page filtered to the keyword
E3	Open product detail	Name, price, Add-to-Cart present
E4	Add to cart, then remove	Item + subtotal shown; cart empties on remove
E5	Register a new account	Lands authenticated (user menu shown)
E6	Login as seeded user / wrong password	Auth toggles; bad password rejected with error

Every E2E test is black-box against the running app and runs on two viewports (Desktop Chrome and Pixel 5 mobile). No test ever places a real payment — PayPal remains in sandbox mode.

5. Test Execution

All suites were executed locally and re-run automatically in CI. The tables below report each test case with its result. Across unit and integration levels, 21 tests pass; the system suite contributes 12 passing checks (6 journeys × 2 viewports).

5.1. Unit Testing

Module: backend/utils/calcPrices.js — run with `npm run test:unit`.

#	Test case	Technique	Expected	Status
UT-1	Empty cart: items 0, flat shipping, no tax	EP	items 0.00, ship 10.00, tax 0.00, total 10.00	PASS
UT-2	Below threshold (50.00)	EP	ship 10.00, tax 7.50, total 67.50	PASS
UT-3	Exactly 100.00 is NOT free shipping	BVA	ship 10.00, total 125.00	PASS
UT-4	100.01 just above qualifies for free shipping	BVA	ship 0.00	PASS
UT-5	Above threshold (150.00): free shipping	EP / decision table	ship 0.00, tax 22.50, total 172.50	PASS
UT-6	Aggregates multiple line items by price × qty	EP	items 60.00, total 79.00	PASS
UT-7	Always returns 2-decimal fixed strings	Output format	matches <code>^\d+\.\d{2}\$</code>	PASS

Module: backend/utils/generateToken.js — tested in isolation with a spy (test double) replacing the Express response.

#	Test case	Expected	Status
UT-8	Signs a verifiable JWT and stores it in a secure, HTTP-only, SameSite=strict cookie	cookie called once; httpOnly + sameSite=strict; JWT verifies with secret; payload userId matches	PASS

```

C:\Master\SVVT\ProShop_SVVT>npm run test:unit

> proshop2@2.0.0 test:unit
> vitest run backend/__tests__/unit

RUN v2.1.9 C:/Master/SVVT/ProShop_SVVT

✓ backend/__tests__/unit/calcPrices.test.js (7)
✓ backend/__tests__/unit/generateToken.test.js (1)

Test Files  2 passed (2)
Tests       8 passed (8)
Start at    23:28:00
Duration    538ms (transform 32ms, setup 0ms, collect 102ms, tests 12ms, environment 0
ms, prepare 155ms)

```

Figure: Vitest output for the unit suite — all tests passing.

5.2. Integration Testing

Integration tests exercise the real Express app (routes → middleware → controllers → Mongoose models) against an in-memory MongoDB, verifying the components work together end to end over HTTP.

Product API

#	Test case	Expected	Status
IT-P1	GET /api/products returns a paginated list	200, products + page = 1	PASS
IT-P2	keyword filters by name (case-insensitive), hit and miss	hit → 1 item, miss → 0 items	PASS
IT-P3	BUG-001 regression: regex metacharacter handled safely	200, array (no 500 crash)	PASS
IT-P4	GET /api/products/:id returns a single product	200, matching _id	PASS
IT-P5	Invalid ObjectId	404	PASS
IT-P6	Valid-but-missing id	404, "not found"	PASS
IT-P7	POST /api/products unauthenticated	401	PASS

User / Auth API

#	Test case	Expected	Status
IT-U1	Register new user, sets jwt cookie, no password leaked	201, email returned, set-cookie jwt=	PASS
IT-U2	Duplicate registration	400, "already exists"	PASS
IT-U3	Login with correct credentials	200, user name returned	PASS
IT-U4	Login with wrong password	401	PASS
IT-U5	GET /profile without a token	401, "not authorized"	PASS

#	Test case	Expected	Status
IT-U6	Password still works after a profile update (regression)	200 on re-login with original password	PASS

```
C:\Master\SVVT\ProShop_SVVT>npm run test:integration
```

```
> proshop2@2.0.0 test:integration
> vitest run backend/__tests__/integration
```

```
RUN v2.1.9 C:/Master/SVVT/ProShop_SVVT
```

```
stderr | backend/__tests__/integration/productRoutes.test.js
Using NodeJS below 20.19.0
```

```
stderr | backend/__tests__/integration/userRoutes.test.js
Using NodeJS below 20.19.0
```

```
✓ backend/__tests__/integration/productRoutes.test.js (7) 1188ms
✓ backend/__tests__/integration/userRoutes.test.js (6) 1120ms
```

```
Test Files 2 passed (2)
```

```
Tests 13 passed (13)
```

```
Start at 23:28:29
```

```
Duration 4.43s (transform 82ms, setup 0ms, collect 1.67s, tests 2.31s, environment
0ms, prepare 151ms)
```

Figure: Vitest output for the integration suite against in-memory MongoDB — all tests passing.

5.3. System (End-to-End) Testing

System tests are black-box Playwright journeys executed against the running application using the Page Object Model. Each journey runs on two device profiles — Desktop Chrome and Pixel 5 (mobile) — for responsiveness coverage.

#	Journey	Expected result	Desktop	Mobile
E1	Home loads with header and a product list	Header + at least one product card visible	PASS	PASS
E2	Filter products by keyword	URL = /search/Airpods; keyword visible in results	PASS	PASS
E3	Open product, add to cart, then remove	Cart shows item + subtotal; empties on remove	PASS	PASS
E4	A new user can register	Authenticated; user menu replaces sign-in link	PASS	PASS
E5	A seeded user can log in	User menu shows the seeded user's name	PASS	PASS
E6	Login with a wrong password is rejected	Stays on login; "Invalid email or password" shown	PASS	PASS

The screenshot displays a Playwright HTML report with a dark theme. At the top, a search bar is on the left, and a summary bar on the right shows: All 12, Passed 12, Failed 0, Flaky 0, Skipped 0, with refresh and settings icons. Below this, the date and time '6/14/2026, 11:14:15 PM' and 'Total time: 11.4s' are shown. The report is organized into four sections, each with a dropdown arrow and a file name: '01-home.spec.js', '02-search.spec.js', '03-product-cart.spec.js', and '04-auth.spec.js'. Each section contains two test entries, one for 'chromium' and one for 'mobile chrome', both marked with a green checkmark and a 'smoke' label. The test descriptions are truncated, showing only the first part of the test name. The duration for each test is listed on the right.

File	Test Name	Viewport	Status	Label	Duration
01-home.spec.js	Home page › loads with header and a product list @smoke	chromium	Passed	smoke	5.3s
	Home page › loads with header and a product list @smoke	Mobile Chrome	Passed	smoke	5.2s
02-search.spec.js	Search › filters products by keyword @smoke	chromium	Passed	smoke	5.4s
	Search › filters products by keyword @smoke	Mobile Chrome	Passed	smoke	5.4s
03-product-cart.spec.js	Product detail & cart › open a product, add to cart, then remove it @smoke	chromium	Passed	smoke	6.2s
	Product detail & cart › open a product, add to cart, then remove it @smoke	Mobile Chrome	Passed	smoke	3.4s
04-auth.spec.js	Authentication › a new user can register @smoke	chromium	Passed	smoke	7.6s
	Authentication › a seeded user can log in @smoke	chromium	Passed	smoke	7.6s

Figure: Playwright HTML report — 12 checks passing across two viewports.

6. Test Coverage Analysis

Coverage was measured with Vitest's v8 provider (`npm run test:coverage`), which generates an HTML and LCOV report under `coverage/`. The strategy deliberately targets the critical business modules — pricing, authentication, routing, and models — which reach full or near-full coverage; payment/upload helpers that require live third-party services are intentionally exercised less.

Module	Line coverage	Branch coverage
utils/calcPrices.js	100% (34/34)	5/5
utils/generateToken.js	100% (17/17)	1/1
models/userModel.js	100% (51/51)	4/4
models/productModel.js	100% (75/75)	—
models/orderModel.js	100% (83/83)	—
routes/*.js (product/user/order)	100%	—
middleware/checkObjectId.js	100% (21/21)	3/3
middleware/asyncHandler.js	100% (4/4)	2/2
middleware/errorMiddleware.js	78.9% (15/19)	1/3
middleware/authMiddleware.js	71.8% (28/39)	3/4
app.js	81.6% (40/49)	0/1
controllers/userController.js	63.5% (115/181)	7/11
controllers/productController.js	49.1% (78/159)	5/5
controllers/orderController.js	28.0% (42/150)	—
utils/paypal.js (sandbox-only)	37.4% (34/91)	—
routes/uploadRoutes.js	42.9% (21/49)	—

Across the backend, roughly 65% of statements are covered overall. The critical, business-logic-bearing modules — pricing, token generation, models, routing, and the ObjectId guard — are fully covered, satisfying the test plan's exit criterion that the critical modules be covered.

All files

65.12% Statements 784/1881 77.5% Branches 31/40 41.17% Functions 7/17 65.12% Lines 784/1881

Press *n* or *j* to go to the next uncovered block, *b*, *p* or *k* for the previous block.

Filter:

File ▲		Statements ▾		Branches ▾		Functions ▾		Lines ▾	
backend		81.63%	40/49	0%	0/1	100%	0/0	81.63%	40/49
backend/config		0%	0/13	0%	0/1	0%	0/1	0%	0/13
backend/controllers		47.95%	235/490	75%	12/16	0%	0/1	47.95%	235/490
backend/middleware		81.48%	66/81	75%	9/12	60%	3/5	81.48%	66/81
backend/models		100%	209/209	100%	4/4	100%	1/1	100%	209/209
backend/routes		71.13%	69/97	100%	0/0	0%	0/3	71.13%	69/97
backend/utlis		59.85%	85/142	100%	6/6	50%	3/6	59.85%	85/142

Figure: HTML coverage report generated by the v8 provider.

7. Bug Reports and Fixes

7.1. BUG-001 — Product search crashes on regex metacharacters (regex injection)

Field	Detail
Severity	High (unhandled HTTP 500 + regex-injection / ReDoS risk)
Component	backend/controllers/productController.js → getProducts
Status	Fixed
Found by	Integration test (productRoutes.test.js "BUG-001 regression"), corroborated by static-analysis review of untrusted input.

Description

The search handler built a MongoDB query by injecting the raw, user-supplied keyword directly into a `$regex` with no escaping. A keyword containing regex metacharacters produced an invalid pattern; MongoDB threw, the async error reached the global error handler, and the request failed with HTTP 500. It was also a regex-injection / ReDoS vector — a crafted pattern can force pathological matching.

Steps to reproduce

1. GET `/api/products?keyword=(` (URL-encoded `%28`). 2. Expected: HTTP 200 with a (possibly empty) product list. 3. Actual (before fix): HTTP 500. Verified by the failing test: "expected 500 to be 200".

Fix

Escape regex metacharacters before constructing the pattern so the keyword is treated as a literal substring match:

```
$regex: req.query.keyword.replace(/[\.*+?^${}()|[\]\\"/g, '\\$&'),
$options: 'i',
```

Verification

After the fix, `npm run test:integration` returns HTTP 200 for the crafted keyword and the full suite is green. The change removes both the crash and the injection vector, and the regression test (IT-P3) permanently guards the behaviour.

7.2. Code-quality fix — password re-hash on profile update

A second defect was guarded with a regression test in the user suite. The Mongoose `pre('save')` hook re-hashed the password on every save; updating an unrelated profile field (e.g. name) re-hashed the already-hashed password, breaking subsequent logins. The hook was corrected to return early when the password is unmodified:

```
userSchema.pre('save', async function (next) {
  if (!this.isModified('password')) return next();
  const salt = await bcrypt.genSalt(10);
  this.password = await bcrypt.hash(this.password, salt);
});
```

```
next();
});
```

Regression test IT-U6 registers a user, updates the profile name, and confirms the original password still logs in (HTTP 200) — failing before the fix, passing after.

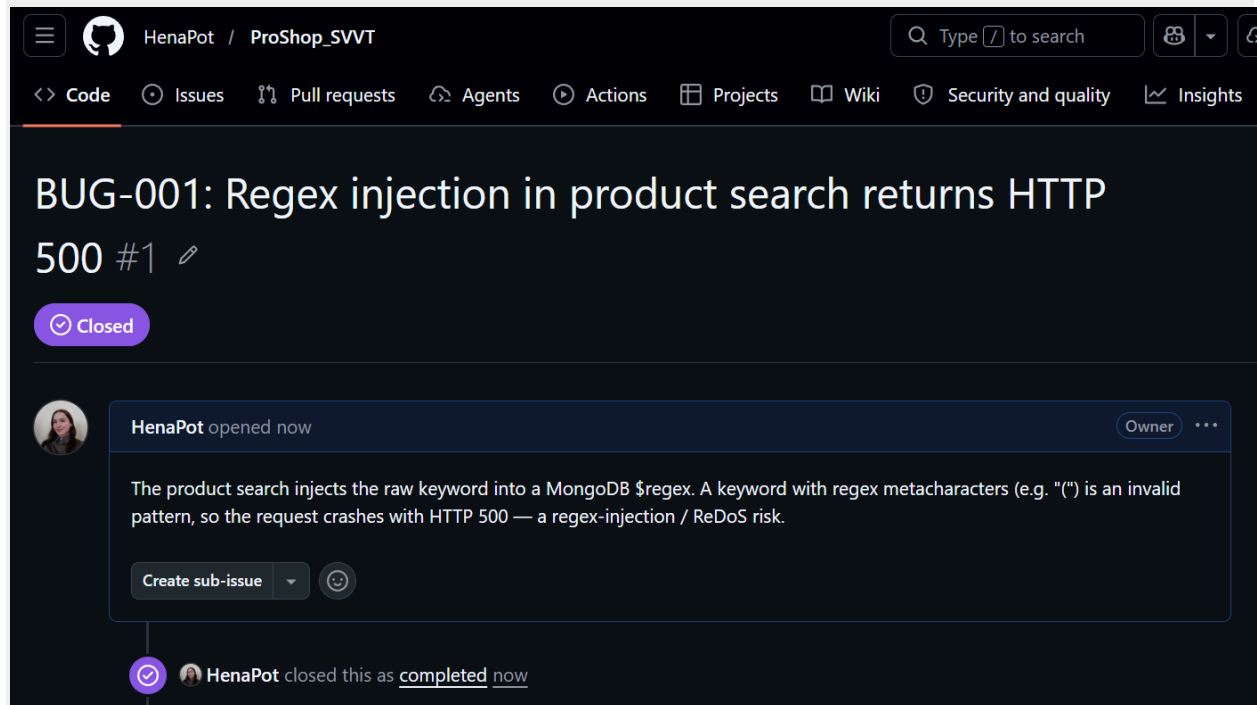


Figure: Bug fix committed and linked to the corresponding issue in the repository.

8. Conclusion

8.1. Testing Summary

Activity	Tool	Count	Result
Static analysis	ESLint 9	8 → 1 issue	0 errors (1 justified warning)
Unit tests	Vitest	8	All passing
Integration tests	Vitest + Supertest	13	All passing
System / E2E tests	Playwright	6 × 2 viewports = 12	All passing
Coverage	Vitest + v8	~65% lines; critical modules 100%	Target met
Documented bugs	BUG-001 + re-hash guard	2 found & fixed	Fixed + regression-tested

In total, 21 unit and integration tests and 12 end-to-end checks pass, static analysis is clean apart from one justified warning, the critical modules are fully covered, and two real defects were found, fixed, and locked down with regression tests — all re-verified automatically in CI.

8.2. Lessons Learned and Final Thoughts

Applying the full verification and validation pipeline to a real, third-party codebase showed how the different testing levels complement one another. Static analysis was not just a style check — it surfaced the untrusted-input path that became BUG-001, demonstrating how non-execution-based verification can reveal genuine security defects. Unit tests with boundary value analysis pinned down the exact behaviour of the free-shipping threshold (strictly greater than 100), the kind of off-by-one rule that is easy to get wrong. Integration tests against an in-memory database gave fast, deterministic confidence in the API contract without depending on external services, and the end-to-end Playwright journeys validated the system the way a real user experiences it, on both desktop and mobile.

The most valuable outcome was the bug-fix-plus-regression-test cycle: each defect was reproduced with a failing test, fixed, and then permanently guarded so it cannot return — and continuous integration makes that guarantee hold on every future change. The main limitation was that real payment processing could not be validated because checkout intentionally stays on the PayPal sandbox; this, together with the upload and PayPal helper modules, accounts for the lower coverage in those non-critical areas. Overall, ProShop is a well-structured application, and the testing effort both verified its core behaviour and measurably improved its quality and security.